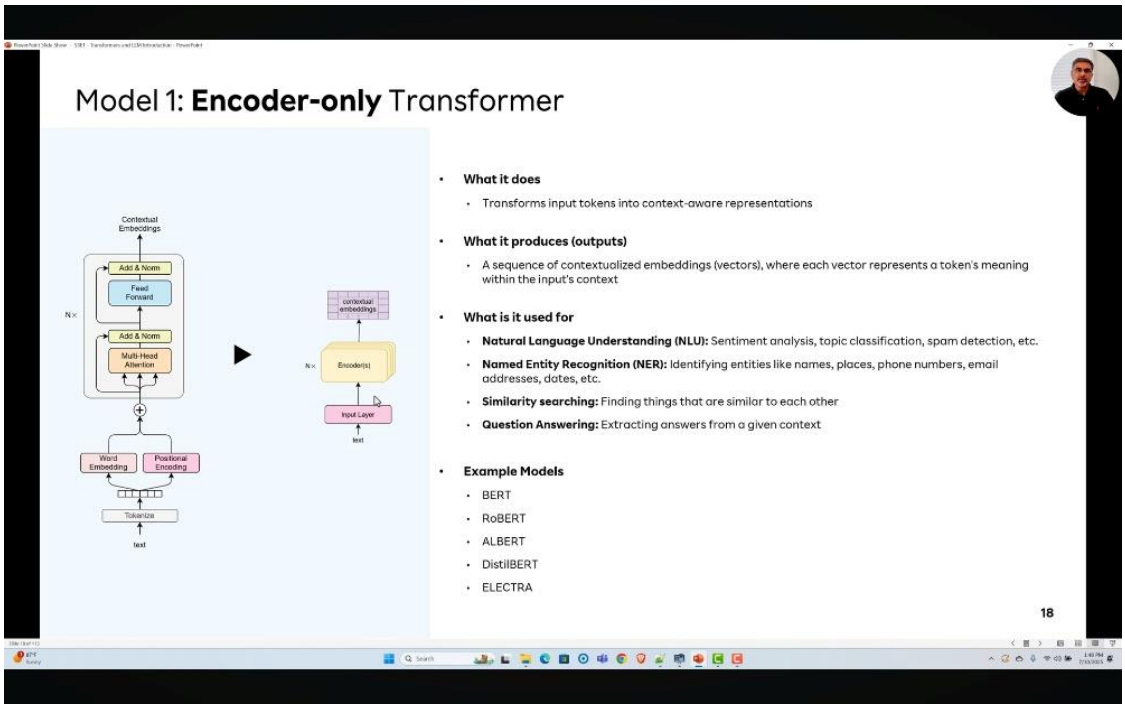
INTRODUCTION

A Transformer is an attention-based neural architecture that models sequence data using stacked multi-head self-attention and feed-forward layers, enabling global context modelling through parallel computation and eliminating the need for recurrence. It forms the foundation of most modern large-scale language models.

A Large Language Model (LLM) is a deep neural network, typically based on the Transformer architecture, trained on massive text corpora using self-supervised learning to model the probability distribution of natural language and generate or understand text.

An encoder-only transformer is a neural architecture composed exclusively of stacked self-attention encoder layers that produce contextualized bidirectional representations of input tokens, typically used for understanding tasks such as classification, tagging, and semantic analysis.



Now learn form 3.1 – 6, 7, 8, 9, 10

(only learn architecture of encoder only transformer right now)

ENCODER ONLY TRANSFORMER

Word Embedding

(already learnt)

Positional Encoding

Positional Encoding (PE) is a technique used in encoder-only Transformers to inject explicit information about the position of each token in the input sequence into the model's input embeddings.

In an encoder-only Transformer, self-attention is permutation-invariant — which means Self-attention does not capture sequence order by itself hence, positional encoding is required to provide word order information, so that the model can distinguish between sequences like:

"dog bites man" vs "man bites dog"

Importance –

1. Make attention position-aware
2. Preserve sequence structure across layers - Position information propagates through all encoder layers via residual connections.

Mathematical Formulation of Positional Encoding

The above formula is used to convert a token's position in a sentence into numerical values so the encoder-only Transformer knows the order of words.

Notation

- Sequence length: N
- Embedding dimension: d_{model}
- Token position: pos
- Embedding dimension index: i

Positional Encoding Formula

For even dimensions:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}}\right)$$

For odd dimensions:

$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}}\right)$$

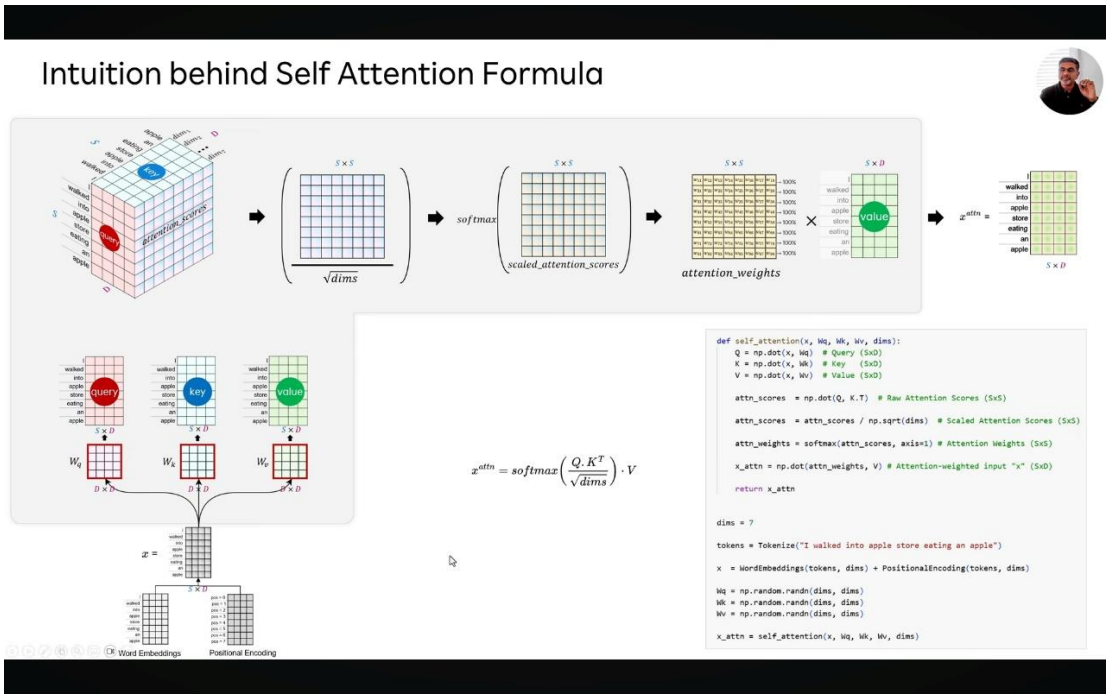
Even dimensions → indices: $0,2,4,6, \ldots$

Odd dimensions → indices: $1,3,5,7, \ldots$

These indices are not token positions. They are feature indices inside the embedding vector.

Self – Attention

Self-attention enables each token to look at all other tokens in the input and gather the most relevant contextual information and also in self-attention, Queries, Keys, and Values all come from the same input sequence.



Intuition –

In the sentence “I walked into the Apple store eating an apple,” the same word “apple” has different meanings depending on surrounding words. Self-attention allows each occurrence of “apple” to look at all other words in the sentence, assign importance to relevant ones like “store” or “eating,” and build a context-aware representation. More generally, self-attention models all relationships between tokens, enabling contextual understanding**.**

Also here,

Example: “The animal didn’t cross the street because it was too tired.”

To understand “it”, the encoder must look at “animal”

Need of Self Attention are –

1. Captures Long-Range Dependencies

- Traditional sequential models struggle with distant relationships
- Self-attention directly connects any token to any other token in one step

2. Contextual Word Representations

- The meaning of a token depends on surrounding tokens
- Self-attention produces context-aware embeddings

3. Parallel Computation

- All tokens attend simultaneously
- Much faster than sequential architectures

Self-Attention Formula (Encoder-Only Transformer)

The scaled dot-product self-attention used inside the encoder is:

$$\boxed{\textbf{Self-Attention}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \textbf{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^\top}{\sqrt{\boldsymbol{d_k}}}\right)\boldsymbol{V}}$$

---

1. Where Do Q, K, V Come From? (Very Important)

Input to the encoder:

$$\boldsymbol{X} = \textbf{Token Embeddings} + \textbf{Positional Encodings}$$

From this input $\boldsymbol{X} \in \mathbb{R}^{\boldsymbol{n} \times \boldsymbol{d}_{\text{model}}}$:

$$\begin{aligned} \boldsymbol{Q} &= \boldsymbol{X}\boldsymbol{W_Q} \\ \boldsymbol{K} &= \boldsymbol{X}\boldsymbol{W_K} \\ \boldsymbol{V} &= \boldsymbol{X}\boldsymbol{W_V} \end{aligned}$$

- **$\boldsymbol{W_Q}, \boldsymbol{W_K}, \boldsymbol{W_V}$are learned projection matrices**
- **Same input → different projections → different roles**

Intuition

- Query (Q): “What am I looking for?”
- Key (K): “What do I contain?”
- Value (V): “What information do I provide?”

---

2. Dot Product: $\boldsymbol{Q}\boldsymbol{K}^\top$

$$\boldsymbol{Q}\boldsymbol{K}^\top$$

- Computes similarity between every query and every key
- Output shape: $n \times n$
- Each entry = how relevant token *j* is to token *i*

Intuition

"How much should token *i* pay attention to token *j* ?"

---

3. Scaling by $\sqrt{d_k}$

$$\frac{\boldsymbol{QK}^\top}{\sqrt{\boldsymbol{d_k}}}$$

Why scaling is needed?

- Dot products grow large when $d_k$is large
- Large values → softmax becomes too peaky
- Scaling stabilizes gradients

Interview Line

Scaling prevents attention scores from becoming excessively large and stabilizes training.

---

4. Softmax: Turning Scores into Weights

$$\textbf{softmax}\left(\frac{\boldsymbol{QK}^\top}{\sqrt{\boldsymbol{d_k}}}\right)$$

- Converts scores into probabilities
- Each row sums to 1
- Higher value → more attention

Intuition

"Which tokens matter the most to me?"

---

5. Weighted Sum with Values (V)

Attention Weights $\times V$

- Combines information from all tokens
- Important tokens contribute more

Output

- Shape: $n \times d_v$
- Each token now has a context-aware representation

---

6. What This Achieves (Encoder Perspective)

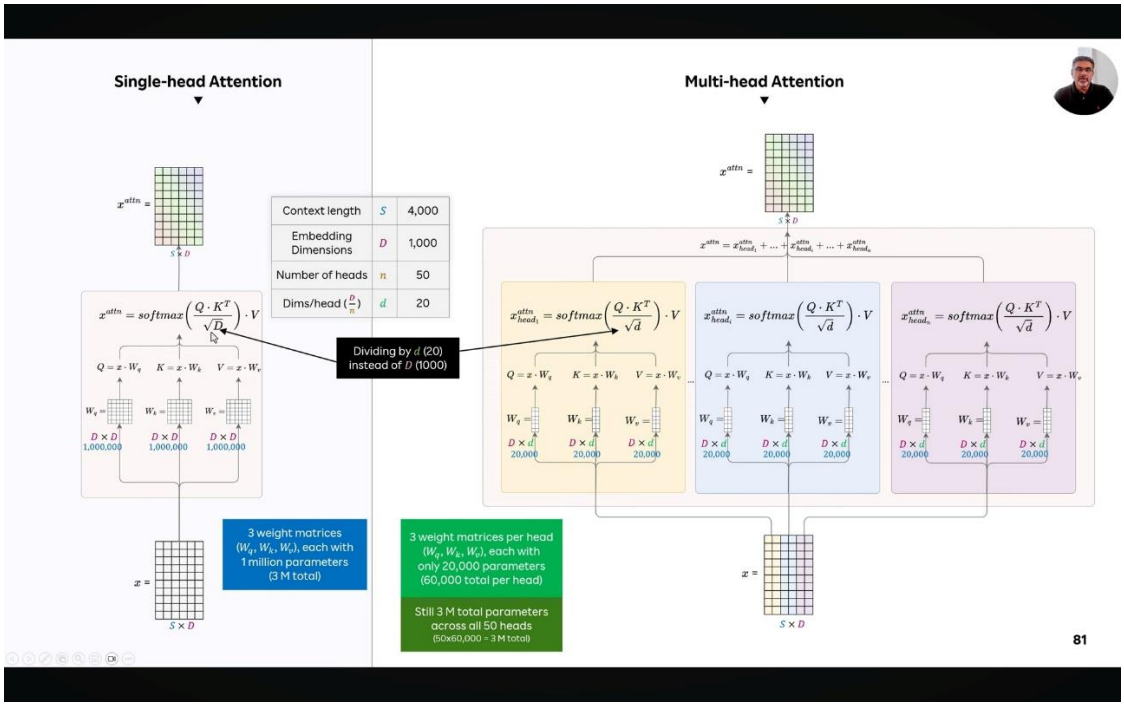
- Each token:
    - Looks at all other tokens
    - Decides relevance dynamically
    - Builds a context-aware embedding

This process is repeated in every encoder layer.

(can take visual reference from 3.1 – 35)

Multi-Head Attention

Multi-head attention splits queries, keys, and values into multiple heads, computes self-attention in parallel for each head, and then concatenates the results to produce richer, context-aware token embeddings.



Intuition (Simple Example)

Sentence:

"I walked into the Apple store eating an apple."

- Head 1 may focus on **company-related context**: "Apple" → "store"
- Head 2 may focus on **food context**: "apple" → "eating"
- Head 3 may focus on **syntax**: "I" → "walked"

Each head learns **different aspects of relationships**, and combining them gives a **richer embedding** for each token.

How Multi-Head Attention Works (Step by Step)

Step 1: Input

$$X = \text{Token Embedding} + \text{Positional Encoding}$$

Step 2: Linear Projections

For each head h:

$$Q_h = XW_Q^h, K_h = XW_K^h, V_h = XW_V^h$$

- Each head has its own learned projection matrices

- Projected dimension: $d_k = d_{\text{model}}/\text{num_heads}$

Step 3: Scaled Dot-Product Attention Per Head

$$\text{head}_h = \text{Attention}(Q_h, K_h, V_h) = \text{softmax}\left(\frac{Q_h K_h^\top}{\sqrt{d_k}}\right) V_h$$

- Each head produces context-aware embeddings from a different subspace

---

Step 4: Concatenate Heads

$$\text{Concat}(\text{head}_1, \ldots, \text{head}_H) \in \mathbb{R}^{n \times d_{\text{model}}}$$

- $H \rightarrow$ number of attention heads
- Each head contributes $d_k = d_{\text{model}}/H$ dimensions

---

Step 5: Final Linear Projection

$$\text{MHA}(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_H) W_O$$

- $W_O \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$
- Brings combined representation back to model dimension

(can look for 3.1 – 37)

Layer Normalization

Layer Normalization (LayerNorm) is a normalization technique that normalizes the features of each token independently across the embedding dimension to stabilize and speed up training.

Why Layer Normalization Is Needed

1. Internal Covariate Shift
    - Feature distributions change across encoder layers
    - Makes training unstable
2. Deep Encoder Stability

    - Encoder-only Transformers stack many layers
    - Without normalization → exploding/vanishing gradients
3. Attention Score Stability
    - Self-attention outputs can vary widely in scale
    - LayerNorm keeps representations well-behaved

Intuition - Layer normalization "rescales" each word's embedding so no feature dominates and all features stay in a healthy range.

Mathematical Formula (Must-Know)

Step 1: Compute Mean

$$\mu = \frac{1}{d_{\text{model}}} \sum_{i=1}^{d_{\text{model}}} x_i$$

Step 2: Compute Variance

$$\sigma^2 = \frac{1}{d_{\text{model}}} \sum_{i=1}^{d_{\text{model}}} (x_i - \mu)^2$$

Step 3: Normalize

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

- $\epsilon \rightarrow$ small constant for numerical stability

---

Step 4: Scale and Shift (Learnable Parameters)

$$\boxed{y_i = \gamma \hat{x}_i + \beta}$$

- $\gamma \rightarrow$ learnable scale

- $\beta \rightarrow$ learnable shift

These allow the model to **recover representational power** if needed.

Feed Forward Layer

Feed-forward layer applies a non-linear transformation to each token's representation independently to increase model capacity and expressiveness.

After self-attention has gathered context from other words, the feed-forward layer processes each token independently to apply non-linear transformations, enabling the model to extract higher-level features and refine each token's contextual representation.

Example phrasing (interview-friendly):

"In the sentence 'I walked into the Apple store,' self-attention gathers context for each word, while the feed-forward network transforms each word's embedding individually, helping the model capture more abstract patterns such as verb tense, semantic roles, or subtle feature interactions."

How FFN Works Mathematically

For a token embedding $x \in \mathbb{R}^{d_{\text{model}}}$:

$$\text{FFN}(x) = W_2 \text{ GELU}(W_1 x + b_1) + b_2$$

Where:

- $W_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$(expansion layer)
- $W_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$(projection back)
- $d_{\text{ff}}$is usually **4× the model dimension**
- GELU is a smooth, non-linear activation (used in BERT)

Residual Connections

Residual connections (also called skip connections) are a technique where the input of a layer is added directly to the output of the layer before applying normalization or further processing.

Why Residual Connections Are Needed (Encoder Context)

1. Vanishing/Exploding Gradients
    - Deep Transformers stack many layers (12, 24, 48, etc.)
    - Gradients may vanish without shortcuts

2. Easier Learning
    - The layer only needs to learn a residual function, i.e., the difference from the input
    - Makes optimization simpler
3. Information Preservation
    - Original input features are preserved and directly passed forward
    - Helps contextual embeddings retain raw token information

Contextual Embeddings

In an encoder-only Transformer, contextual embeddings are the final output representations for each token, which encode not just the token itself but also its relationships with all other tokens in the sequence.

How Contextual Embeddings Are Generated

Step 1: Input Representation

$$X = \text{Token Embedding} + \text{Positional Encoding}$$

Step 2: Encoder Layer Processing

For each encoder layer (L layers):

1. Self-Attention: Each token attends to all other tokens to gather relevant context

$$Z_{\text{attn}} = \text{SelfAttention}(X)$$

2. Residual + LayerNorm: Preserve original features and stabilize training

$$Z_1 = \text{LayerNorm}(X + Z_{\text{attn}})$$

3. Feed-Forward Layer: Apply non-linear transformation to each token

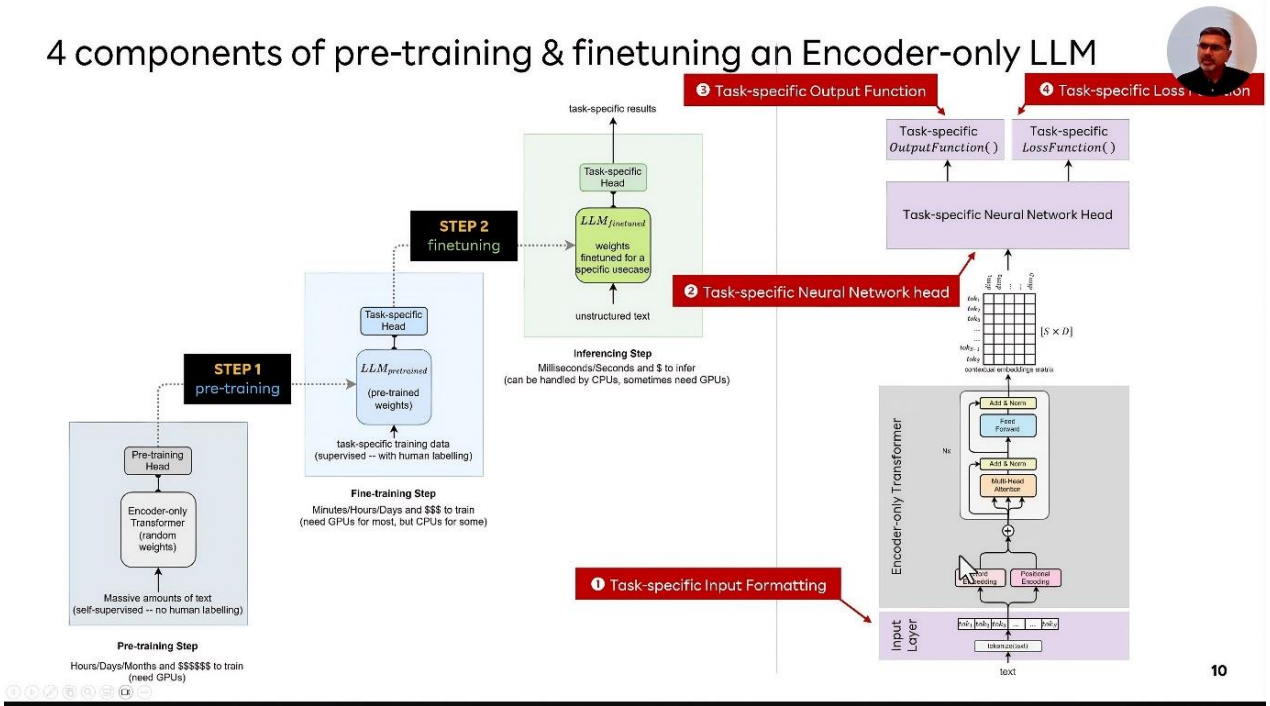
$$Z_{\text{ffn}} = \text{FFN}(Z_1)$$

4. Residual + LayerNorm Again:

$$X_{\text{next}} = \text{LayerNorm}(Z_1 + Z_{\text{ffn}})$$

Repeat this for all encoder layers. After L layers, each token embedding contains context from the entire sequence.

PRETRAINING OF ENCODER ONLY TRANSFORMER



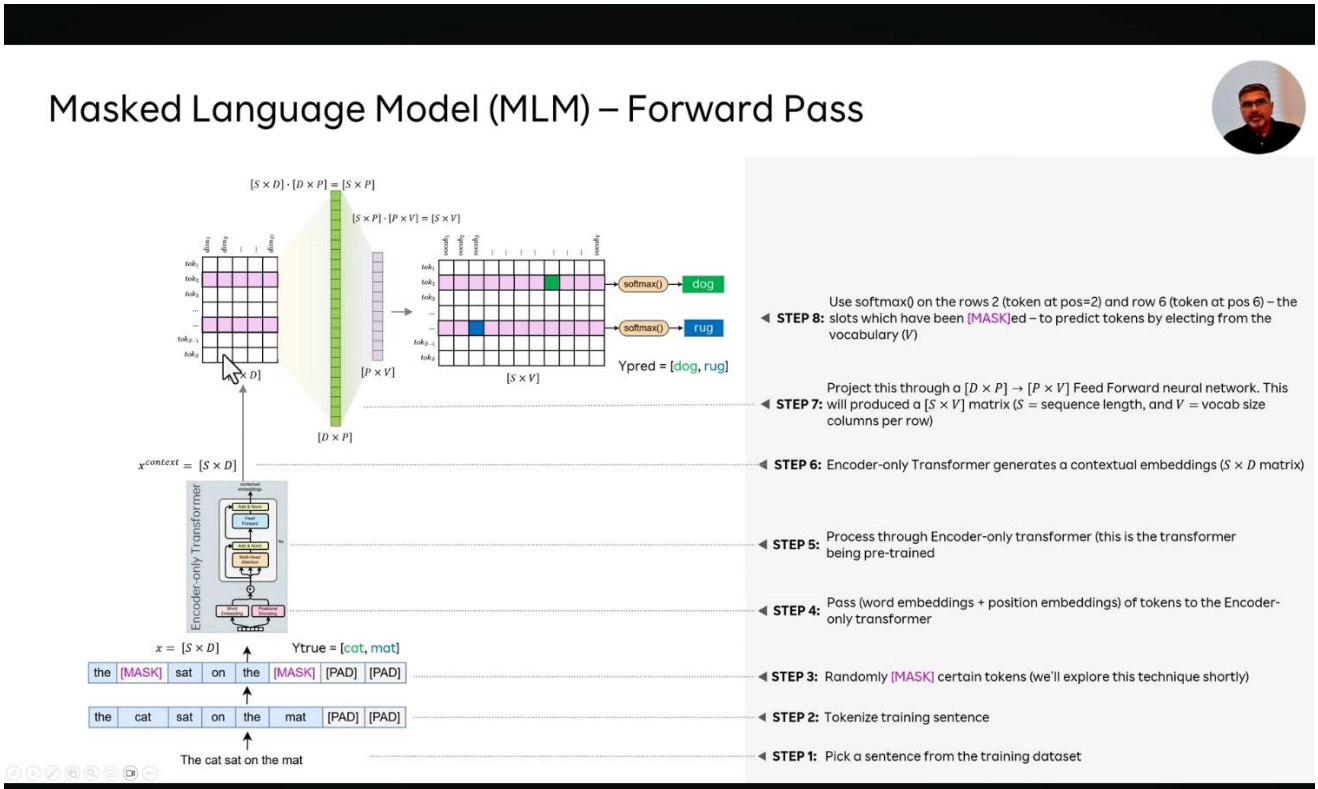
Various pretraining techniques are –

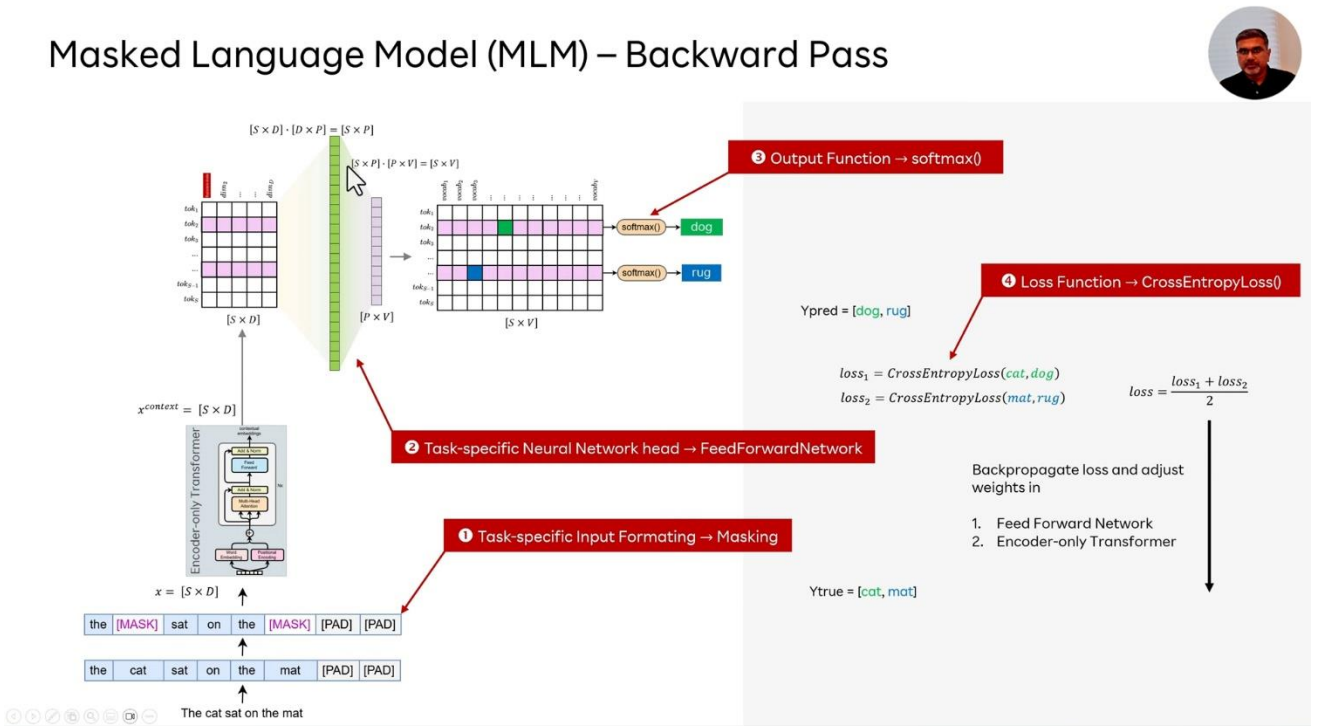
1. MLM (Masked Language Model)
2. RTD (Replaced Token Detection)
3. NSP (Next Sentence Prediction)
4. SOP (Sentence Order Prediction)

MLM (Masked Language Model)

Masked Language Modeling (MLM) is a self-supervised learning technique where some tokens in a sentence are randomly masked, and the model is trained to predict the original tokens using the surrounding context.

Working steps -





How to determine which tokens to [MASK] ?

Step 1: Random Selection

Select ~15% of tokens uniformly at random from the input sequence.

Step 2: Apply Masking Strategy

For each selected token:

80% → Replace with [MASK]

10% → Replace with random token

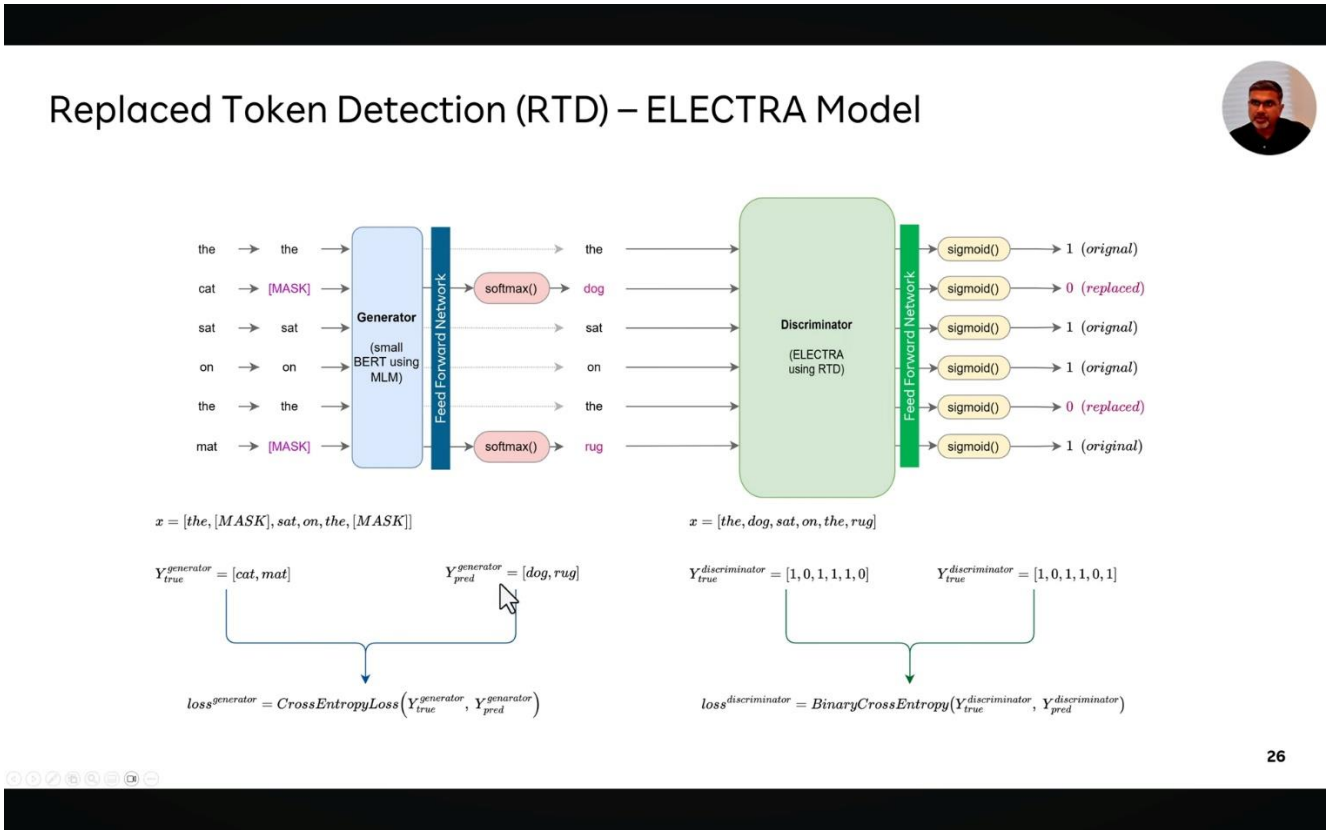
10% → Keep unchanged

RTD (Replaced Token Detection)

RTD trains a model to detect replaced tokens in a sequence, enabling efficient and fully supervised learning at every token position.

RTD (Replaced Token Detection) – Working Steps

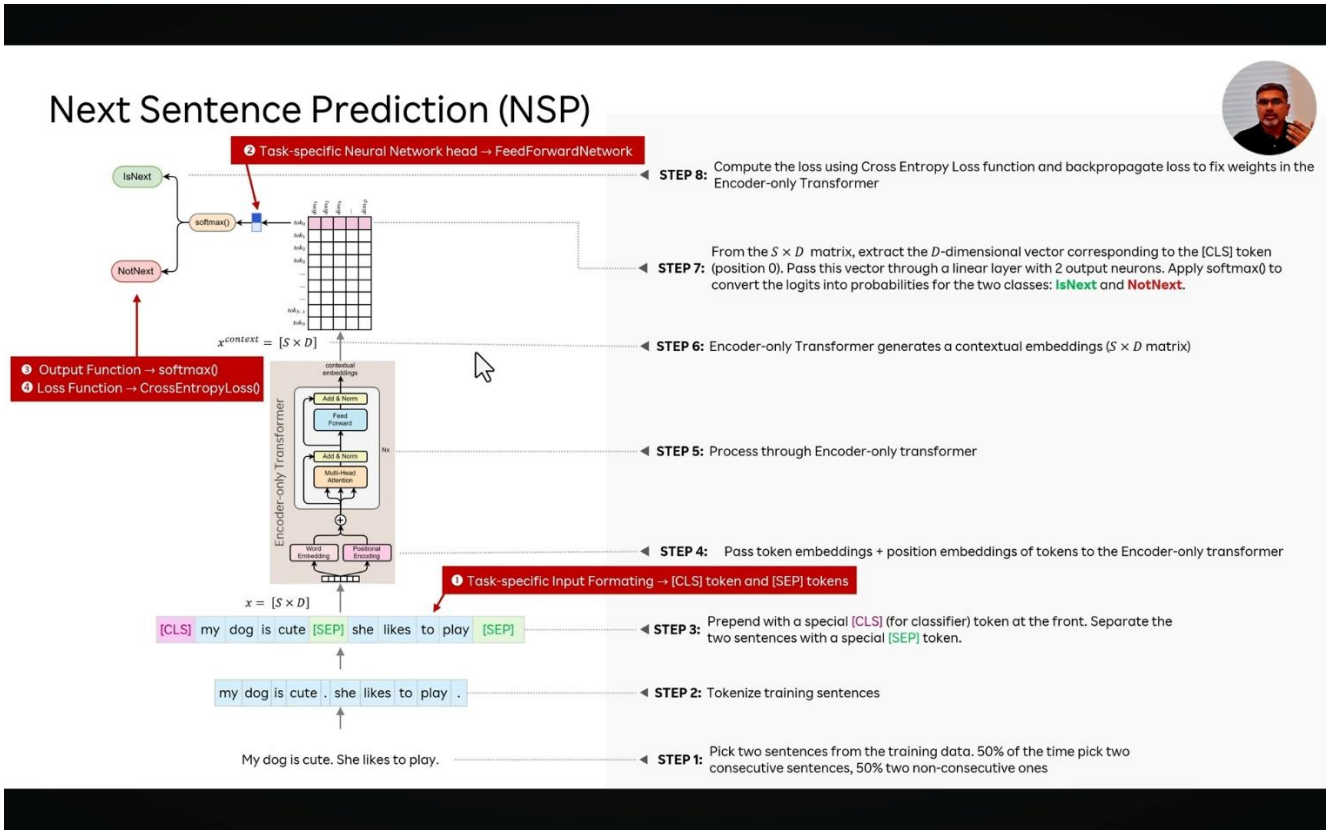
1. **Input Sentence** - A raw sentence is taken as input for processing.
2. **Tokenization** - The sentence is tokenized into subword tokens with special tokens like [CLS] and [SEP].
3. **Masking Tokens** - Some tokens are randomly selected and replaced with [MASK] tokens.
4. **Generator Prediction** - A small generator model predicts possible tokens for the masked positions.
5. **Token Replacement** - Masked tokens are replaced with generator-predicted tokens (correct or incorrect).
6. **Corrupted Sequence** - A new sequence is formed containing both original and replaced tokens.
7. **Discriminator Input** - The corrupted sequence is fed into the discriminator model.
8. **Token-Level Classification** - The discriminator predicts for each token whether it is original or replaced.
9. **Loss Computation** - Binary classification loss is computed for each token (real vs fake).
10. **Backpropagation** - The discriminator (and sometimes generator) weights are updated to improve detection.



NSP (Next Sentence Prediction)

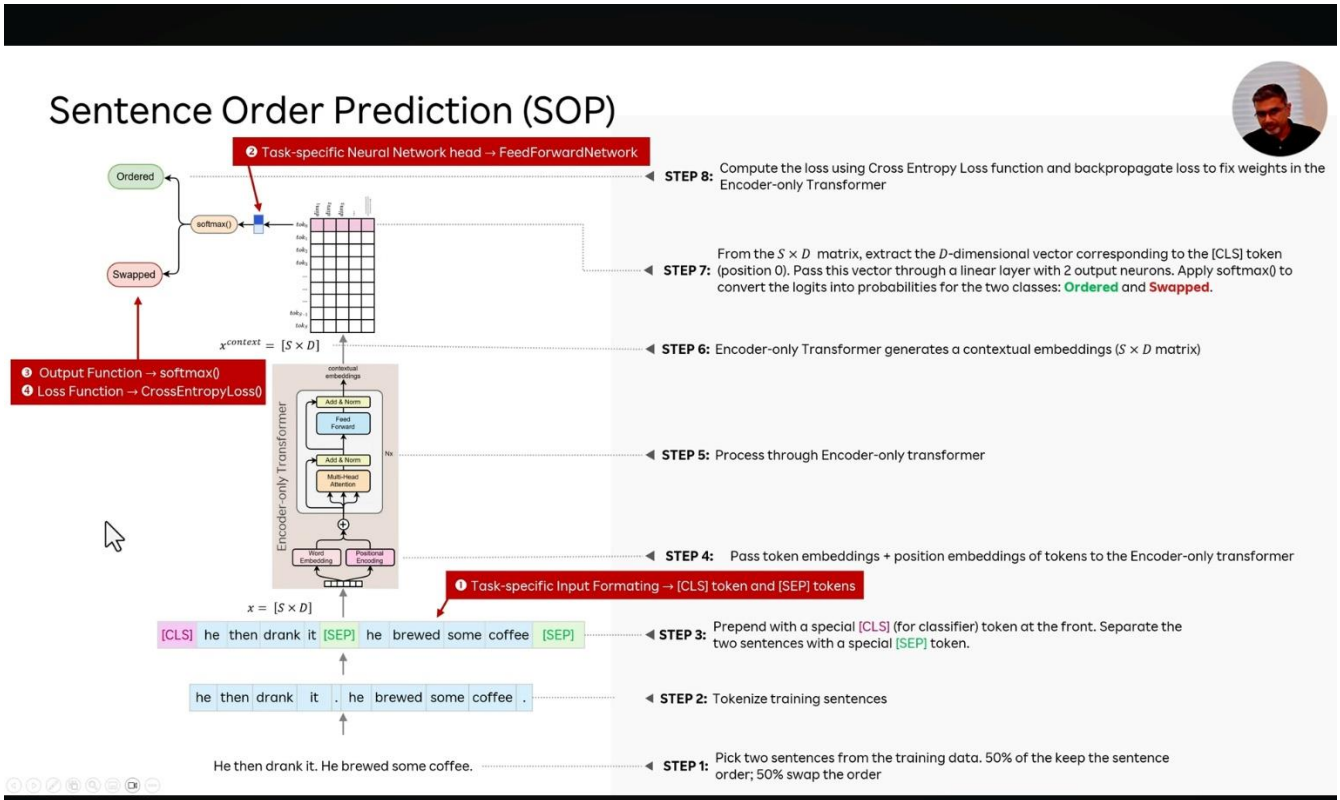
NSP trains a model to determine whether two sentences are sequentially related or randomly paired.

Working –



SOP (Sentence Order Prediction)

SOP is a binary task that trains a model to detect whether two consecutive text segments are in the correct order or swapped, improving discourse-level understanding.



COMPARISON OF VARIOUS ENCODER ONLY TRANSFORMER MODELS

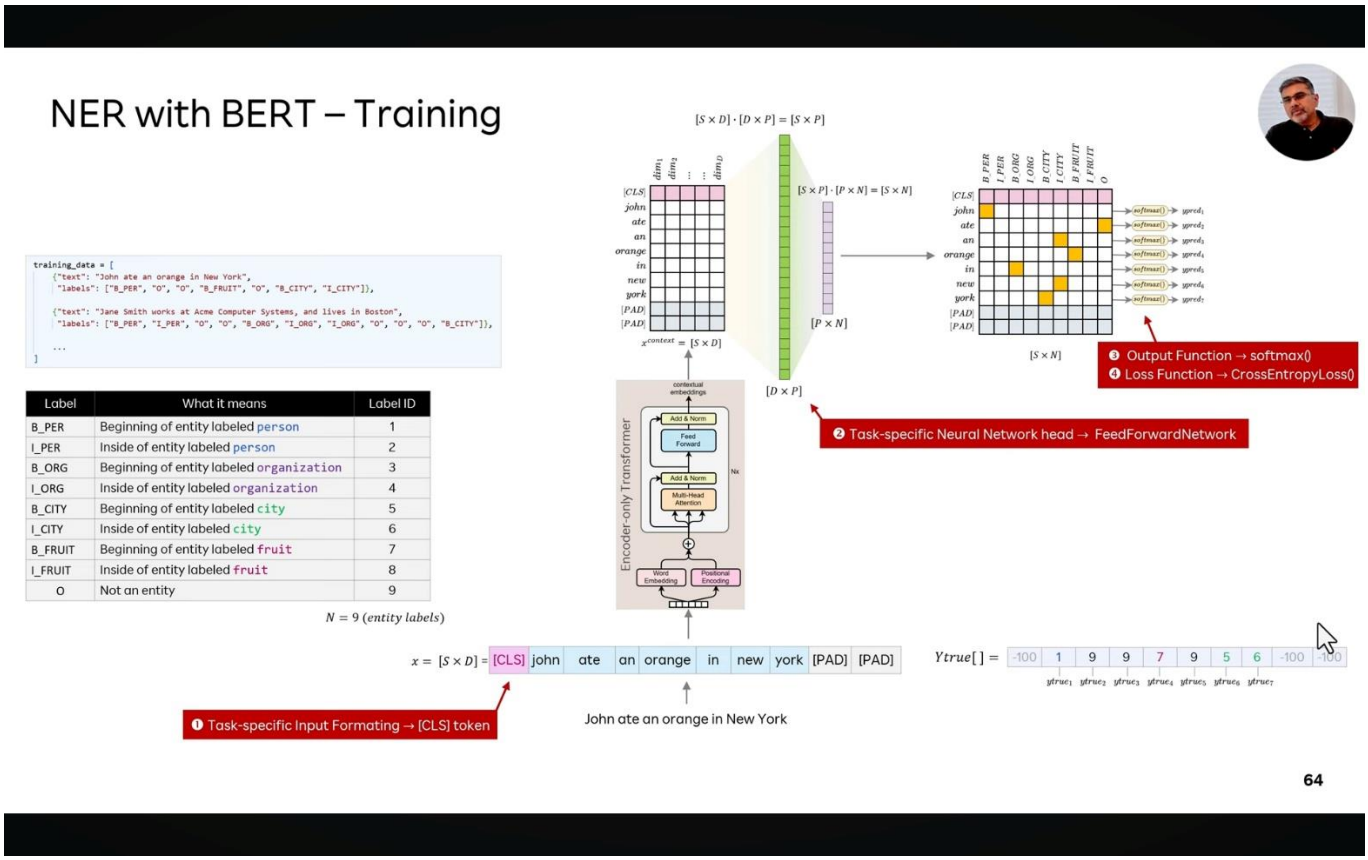
Comparison of different pre-trained BERT models

Model Name	MLM	RTD	NSP	SOP	Model Size (Parameters)	Training Dataset Type & Size	Vocab Size	Year Introduced, Institution	HuggingFace Model
BERT	✓	×	✓	×	Base: 110M Large: 340M	BookCorpus (800M words) + English Wikipedia (2.5B words) (Total ~3.3 Billion words)	30,522	2018 Google	`bert-base-uncased`
RoBERTa	✓	×	×	×	Base: 125M Large: 355M	Significantly larger dataset (160 GB of text) including: BookCorpus, English Wikipedia, CC-News, OpenWebText, Stories	50,265	2019 Facebook	`roberta-base`
ALBERT	✓	×	×	✓	Base: 12M Large: 18M XXLarge: 235M	BookCorpus + English Wikipedia (similar to BERT)	30,000	2019 Google	`albert-base-v2`
ELECTRA	✓	✓	×	×	Small: 14M Base: 110M Large: 335M	BookCorpus + English Wikipedia / OpenWebText (similar scale to BERT, but highly efficient training)	30,522	2020 Google	`google/electra-base-discriminator`
DistilBERT	✓	×	×	×	Base: 66M (60% smaller than BERT-base)	Distilled from BERT on BERT's original corpus (BookCorpus + English Wikipedia)	30,522	2019 Hugging Face	`distilbert-base-uncased`
DeBERTa	✓	×	×	×	Base: 140M Large: 390M v3 Large: 278M	Large-scale text corpus (~160GB of text) including Wikipedia, BookCorpus, OpenWebText, CC-News. Some variants trained on ~80GB or ~400GB for specific languages.	50,265	2020 (v3 in 2021) Microsoft	`microsoft/deberta-base`
TinyBERT	✓	×	×	×	4M-15M Very small parameter counts via distillation	Distilled from BERT, utilizing BERT's original training corpus. Not pre-trained from scratch on a new corpus.	30,522	2020 Huawei	`huawei-noah/TinyBERT_General_4L_312D`

NER

NER is a sequence labeling task where each token in a sentence is classified into entity categories like person, location, or organization.

It uses BIO (Beginning Inside Outside) tags for the identification of the entities.



CRF (Conditional Random Field) is a probabilistic model used for sequence labeling tasks in Natural Language Processing.

Why to add CRF on top of BERT for NER?

Because CRF captures dependencies between labels and ensures valid sequences.

How would you improve NER performance?

1. Better preprocessing
2. Domain-specific fine-tuning
3. Data augmentation
4. Hyperparameter tuning
5. Error analysis

SIAMESE NETWORK VS BI-ENCODER VS SENTENCE TRANSFORMER

1. Siamese Network (Training Concept)

A Siamese network is a training framework where two or more identical models with shared weights process different inputs, and the model is trained using a similarity-based objective (e.g., contrastive or triplet loss) to learn an embedding space.

What this implies:

Focus = how the model is trained

Goal = learn similarity / distance

Not tied to NLP only (used in vision, speech, etc.)

2. Bi-Encoder (Architecture / Inference Concept)

A bi-encoder is a model architecture where two inputs are encoded independently using the same encoder into fixed embeddings, and their relationship is computed using a similarity function (e.g., cosine similarity), enabling efficient large-scale comparison.

What this implies:

Focus = how the model is used

Goal = fast retrieval / matching

Typically used at inference time

3. Sentence Transformer (Specific Model)

A Sentence Transformer (e.g., Sentence-BERT) is a bi-encoder model built on transformers like BERT that is trained using a Siamese or triplet setup to produce semantically meaningful sentence embeddings optimized for similarity tasks.

What this implies:

Combines both Siamese training and Bi-encoder architecture

Goal = high-quality embeddings out-of-the-box

Objectives

- Siamese Network → To learn an embedding space where similar inputs are close and dissimilar ones are far apart, enabling similarity-based applications like verification, matching, and duplicate detection.
- Bi-Encoder → To independently encode inputs into comparable embeddings for scalable similarity computation, enabling efficient large-scale retrieval and recommendation systems.
- Sentence Transformer (SBERT) → To generate semantically meaningful sentence embeddings optimized for similarity learning, enabling accurate semantic search, clustering, and RAG applications.

Why Sentence Transformers Exist ?

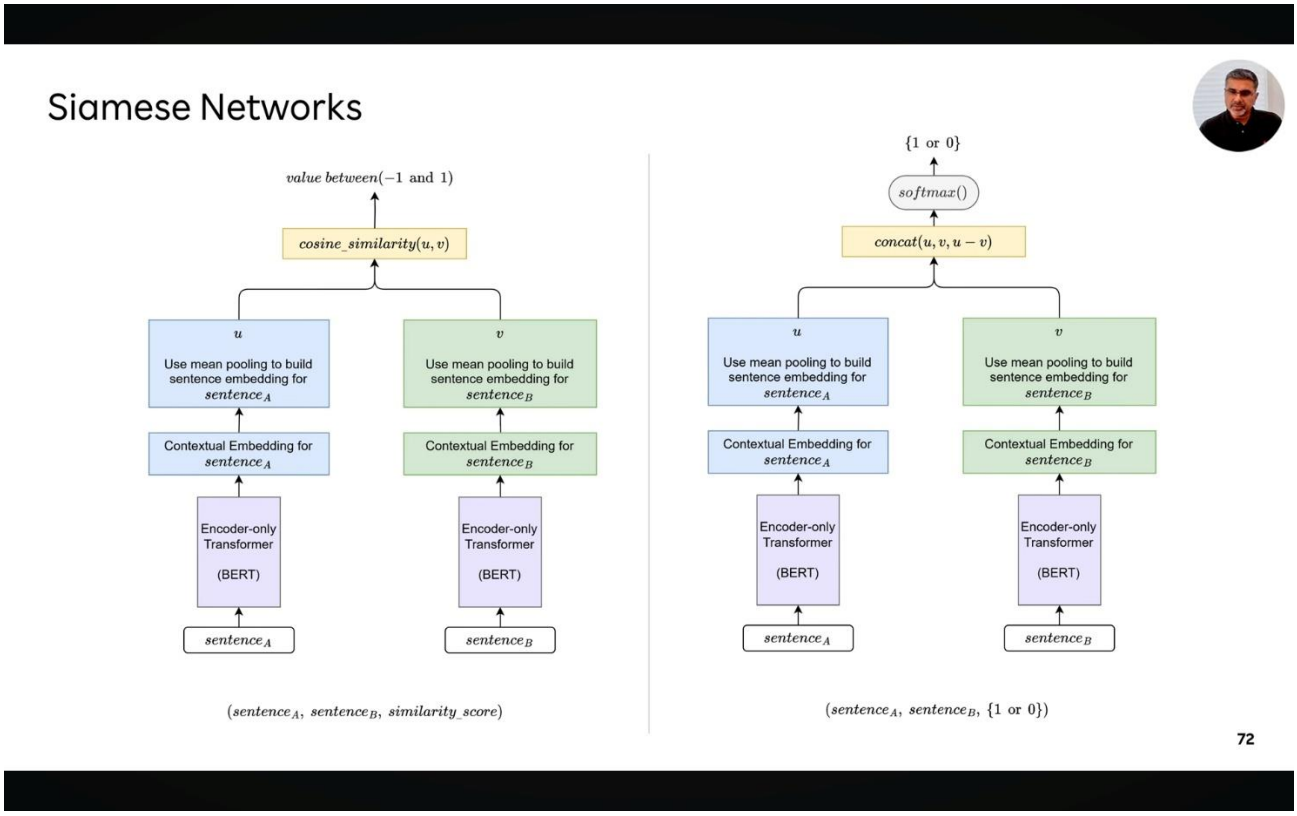
Vanilla BERT is not suitable for sentence similarity tasks directly because:

- It produces token-level embeddings, not sentence-level
- Using [CLS] embedding without fine-tuning is weak for semantic similarity
- Pairwise comparison with BERT is computationally expensive ($O(N^2)$)

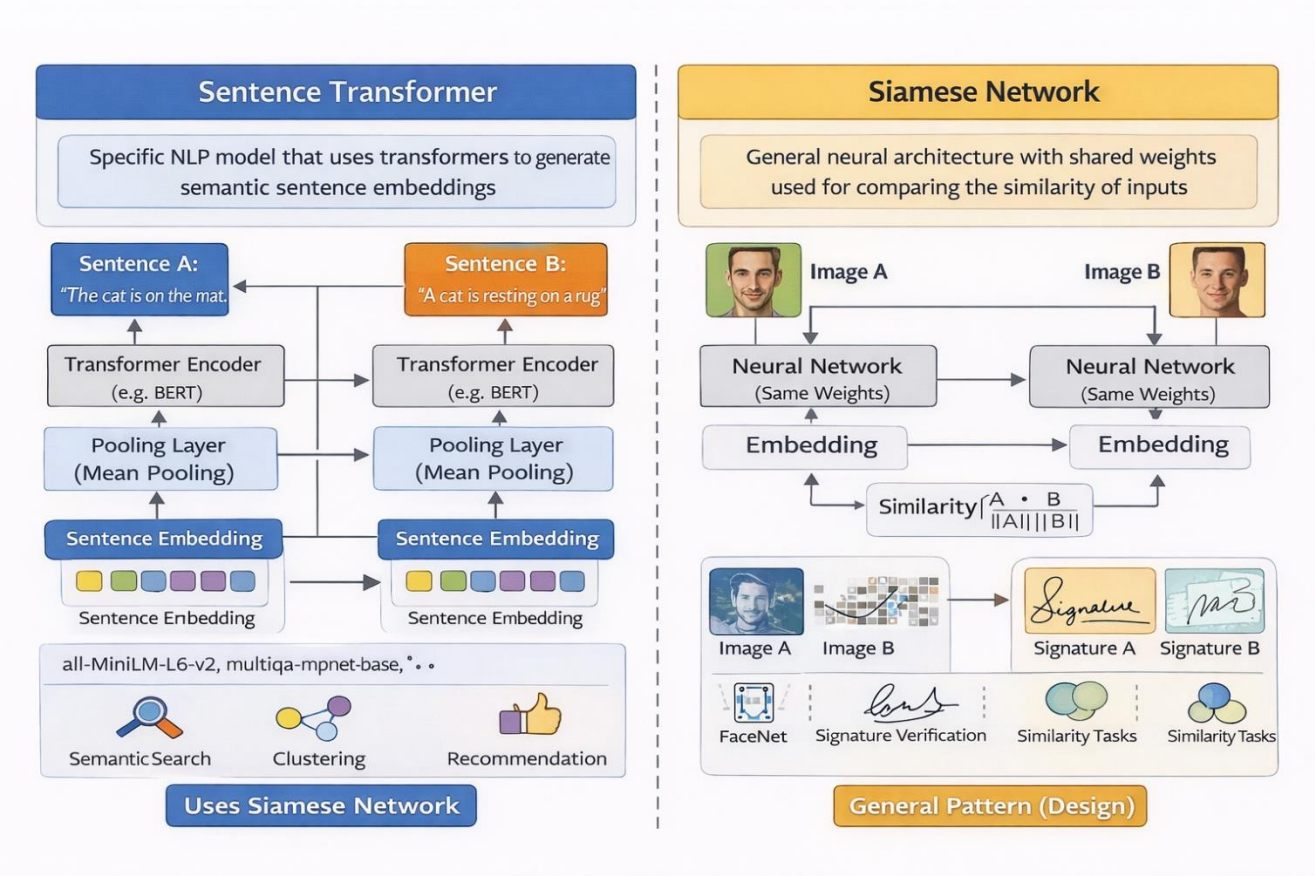
Sentence Transformers solve this by:

- Producing meaningful sentence embeddings
- Enabling efficient similarity search ($O(N)$) using vector indexing)

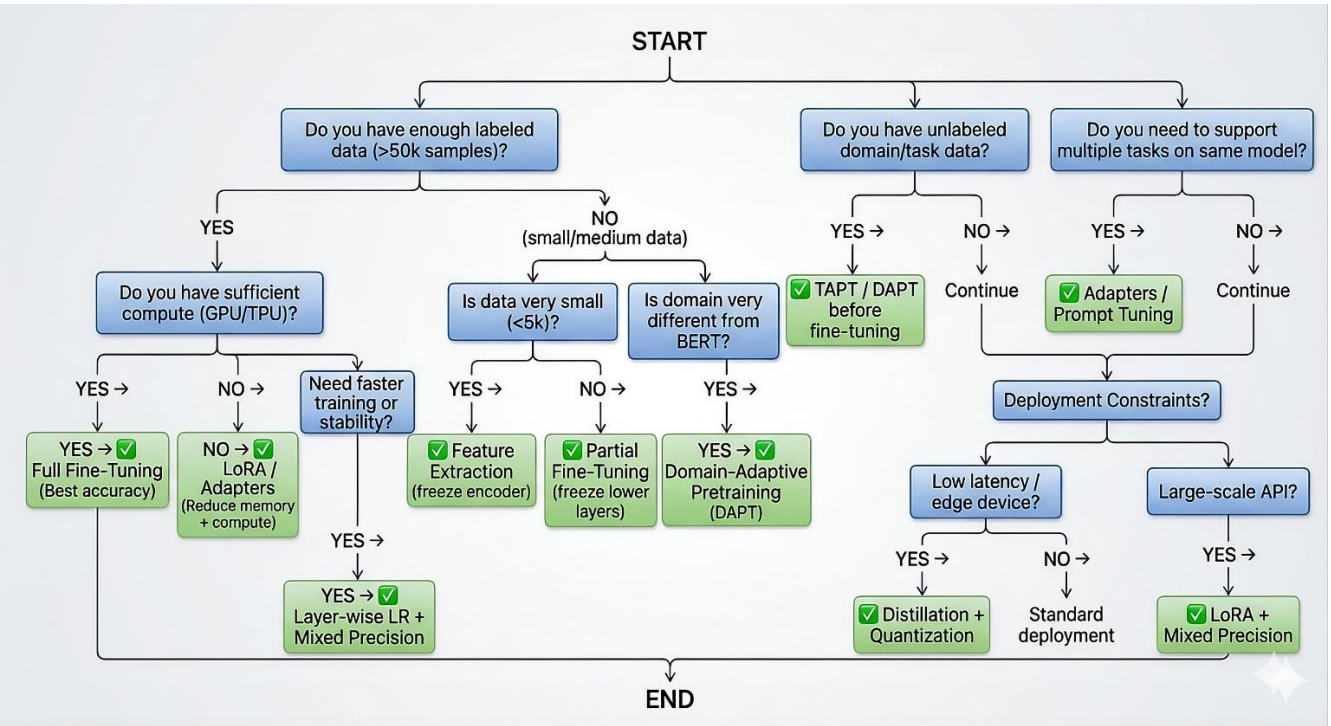
Working steps are –



Above image is actually a sentence transformer (SBERT), not a Siamese network



FINE TUNING OF ENCODER ONLY TRANSFORMERS



In above image, Continue = skip this branch, move forward in decision flow”